MACHINE LEARNING

Mid Term Exam Preparation Guide

Comprehensive Study Material

Table of Contents

1. Introduction to Machine Learning
2. Python Fundamentals for ML
3. Scikit-Learn (sklearn) Library
4. Feature Extraction Techniques
5. Naive Bayes Classifier
6. K-Nearest Neighbors (KNN)
7. One-R Classifier
8. Flask Web Applications with ML
9. Complete Code Examples
10. Practice Problems & Solutions

1. Introduction to Machine Learning

1.1 What is Machine Learning?

Machine Learning is an application of Artificial Intelligence (AI) that enables systems to learn and improve from experience without being explicitly programmed. It is the capability of a machine to replicate intelligent human behavior and make predictions from data.

Key Definitions:

- **Machine Learning:** A subfield of AI that makes systems learn from data patterns
- **Training:** The process of feeding data to a model so it can learn patterns
- **Prediction:** Using the learned model to make decisions on new, unseen data

1.2 Why Machine Learning Now?

- **Large Digital Data:** We now have access to massive amounts of data from the internet, sensors, and digital devices
- **Increasing Computing Power:** Modern computers can process data much faster than before
- **Growth of Algorithms:** New and improved ML algorithms are constantly being developed
- **Industry Support:** Companies are investing heavily in ML research and applications

1.3 Applications of Machine Learning

Application	Description
Image Recognition	Identifying objects, faces, or patterns in images (e.g., facial recognition, medical imaging)
Speech Recognition	Converting spoken words to text (e.g., Siri, Alexa, Google Assistant)
Traffic Prediction	Predicting traffic conditions based on historical data (e.g., Google Maps)
Product Recommendations	Suggesting products based on user behavior (e.g., Amazon, Netflix)
Self-Driving Cars	Autonomous vehicles that can navigate without human input

1.4 Types of Machine Learning

A. Supervised Learning

In supervised learning, the model learns from labeled data. Each training example has an input (features) and a known output (label).

- **Classification:** Predicting a category or class (e.g., spam vs. not spam, disease vs. healthy)
- **Regression:** Predicting a continuous numerical value (e.g., house price, temperature)

B. Unsupervised Learning

The model learns from unlabeled data and finds patterns without explicit guidance.

- **Clustering:** Grouping similar data points together (e.g., customer segmentation, image compression)

1.5 Datasets in Machine Learning

Structured vs Unstructured Data

Aspect	Structured Data	Unstructured Data
Definition	Highly organized, formatted data in tables/databases	No predefined format or organization
Examples	Excel sheets, SQL databases, CSV files	Images, text documents, videos, audio files

2. Python Fundamentals for ML

2.1 Introduction to Python

Python is a high-level general-purpose programming language. Its design philosophy emphasizes code readability with significant indentation. It's the most popular language for machine learning due to its simplicity and powerful libraries.

2.2 Data Structures

A. List

Lists are ordered, mutable collections that can contain items of different types.

Example:

```
w = ["car", 2, 4.6]
print(w[0])  # Output: car
w.append("Cycle")
w.remove("car")
```

B. Dictionary

Dictionaries store data in key-value pairs. As of Python 3.7, dictionaries are ordered.

```
dic = {"st1": "Ali", "st2": "Usman"}
print(dic['st2'])  # Output: Usman
print(dic.keys())  # All keys
```

C. Set

Sets are unordered, unchangeable collections that do not allow duplicate values.

```
myset = {4, 7, 2, 4, 5}
print(myset)  # Output: {2, 4, 5, 7} (no duplicates)
```

D. Tuple

Tuples are ordered, unchangeable collections that allow duplicate values.

```
thistuple = ("apple", "banana", "cherry")
# Tuples cannot be modified after creation
```

2.3 Control Flow

Conditionals (if-elif-else)

```
a = 200
b = 33
if b > a:
    print("b is greater")
elif a == b:
    print("equal")
else:
    print("a is greater")
```

Loops (for and while)

```
# For loop with range
for x in range(6):
    print(x)  # Output: 0 1 2 3 4 5

# While loop
i = 1
while i < 6:
    print(i)
    i += 1
```

2.4 Functions and Classes

Functions

```
def my_function(fname):
    print(fname + " Refsnes")

my_function("Emil")  # Output: Emil Refsnes
```

Classes

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def print_fun(self):
        print(self.name, self.age)

p1 = Person("Usman", 30)
p1.print_fun()  # Output: Usman 30
```

Note: The __init__() function is always executed when the class is instantiated. The 'self' parameter represents the current instance of the class.

3. Scikit-Learn (sklearn) Library

Scikit-learn is the most popular Python library for machine learning. It provides simple and efficient tools for data analysis and modeling.

3.1 One-Hot Encoder

One-Hot Encoder converts categorical features into a binary (0 or 1) numeric array. This is essential because machine learning algorithms require numerical input.

Example:

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

# Original data
data = [['Sunny'], ['Windy'], ['Sunny']]

# Apply One-Hot Encoding
encoder = OneHotEncoder()
encoded = encoder.fit_transform(data).toarray()

# Result: [[1, 0], [0, 1], [1, 0]]
# Sunny = [1, 0], Windy = [0, 1]


Using Get Dummies:
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder
data=[['sunny'],['windy'],['sunny']]

# data=pd.DataFrame(data,columns=['Weather'])

# data=pd.get_dummies(data)
# print(data)
```

3.2 Label Encoder

Label Encoder converts categorical labels into numeric format (0, 1, 2, ...). Used for the target variable in classification.

```
from sklearn.preprocessing import LabelEncoder

labels = ['Play', 'Not Play', 'Play']

encoder = LabelEncoder()
encoded_labels = encoder.fit_transform(labels)

# Result: [1, 0, 1]
# Not Play = 0, Play = 1
```

3.3 Train-Test Split

Splits data into training set (to train the model) and testing set (to evaluate performance). Typically 70-80% train, 20-30% test.

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

# test_size=0.3 means 30% for testing, 70% for training
# random_state ensures reproducibility
```

3.4 Accuracy Score

Accuracy measures the percentage of correct predictions.

```
from sklearn.metrics import accuracy_score

# After training and making predictions
accuracy = accuracy_score(Y_test, Y_predicted)
print(f'Accuracy: {accuracy * 100}%')

# Formula: Accuracy = Correct Predictions / Total Predictions
```

3.5 Common Sklearn Classifiers

Classifier	Import Statement
Naive Bayes	`from sklearn.naive_bayes import GaussianNB`
KNN	`from sklearn.neighbors import KNeighborsClassifier`
SVM	`from sklearn.svm import SVC`

4. Feature Extraction Techniques

Feature extraction transforms raw data into numerical features that machine learning algorithms can process. It reduces the number of features while preserving important information.

4.1 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF measures how important a word is in a document relative to a collection of documents.

Formula:

- **TF (Term Frequency):** How often a word appears in a document
- **IDF (Inverse Document Frequency):** How rare the word is across all documents
- **TF-IDF = TF × IDF**

4.2 Bag of Words (BoW)

Bag of Words converts text into fixed-length vectors by counting word occurrences. Word order is ignored.

Step-by-Step Example:

Step 1: Create Vocabulary

Sentence 1: "He is moving to school"

Sentence 2: "She is not moving to college"

Vocabulary: ["He", "is", "moving", "to", "school", "She", "not", "college"]

Step 2: Create Feature Vectors

Sentence	He	is	moving	to	school	She	not	college
Sentence 1	1	1	1	1	1	0	0	0

Sentence 2	0	1	1	1	0	1	1	1

4.3 N-Gram Model

N-grams capture sequences of words rather than individual words. This preserves some context.

- **Bi-gram (2-gram):** Pairs of consecutive words
- **Tri-gram (3-gram):** Triplets of consecutive words

Example: "I love machine learning"

- Bi-grams: "I love", "love machine", "machine learning"
- Tri-grams: "I love machine", "love machine learning"

4.4 Word Embeddings

Word embeddings represent words as dense vectors in a continuous space where similar words are close together. Popular methods include Word2Vec, GloVe, and FastText.

Example: "king" - "man" + "woman" ≈ "queen"

4.5 Local Binary Pattern (LBP) - Image Feature Extraction

LBP is used to extract texture features from images. It compares each pixel with its neighbors in a 3×3 window.

Algorithm:

1. Select a 3×3 window (center pixel + 8 neighbors)
2. Compare each neighbor with the center pixel
3. If neighbor ≥ center: write 1, else write 0
4. Convert the 8-bit binary number to decimal

Applications:

- Face detection and recognition
- Object detection
- Facial expression recognition
- Remote sensing

5. Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It's called 'naive' because it assumes that all features are independent of each other.

5.1 Bayes' Theorem

Formula:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

- **P(A|B):** Probability of A given B (posterior probability)
- **P(B|A):** Probability of B given A (likelihood)
- **P(A):** Probability of A (prior probability)
- **P(B):** Probability of B (evidence)

5.2 Why 'Naive'?

The classifier assumes class conditional independence, meaning the effect of one feature on the class is independent of other features. This simplifies calculations but may not always be realistic.

5.3 Complete Numerical Example (FROM YOUR PDF)

Problem Statement: Train a Naive Bayesian classifier to predict employee status (Senior or Junior) based on department, age, and salary.

Dataset:

Department	Age	Salary	Status
sales	31-35	46K-50K	**senior**
sales	26-30	26K-30K	**junior**
sales	31-35	31K-35K	**junior**
systems	21-25	46K-50K	**junior**
systems	31-35	66K-70K	**senior**

Test Instance: (sales, 31-35, 66K-70K) - Predict the status?

Step 1: Calculate Prior Probabilities

Count occurrences of each class:

- Total records = 11
- Senior count = 5
- Junior count = 6

P(Senior) = 5/11 = 0.4545

P(Junior) = 6/11 = 0.5455

Step 2: Calculate Conditional Probabilities

For Senior (5 records):

- P(sales|Senior) = 1/5 = 0.2
- P(31-35|Senior) = 2/5 = 0.4
- P(66-70K|Senior) = 2/5 = 0.4

For Junior (6 records):

- P(sales|Junior) = 2/6 = 0.333
- P(31-35|Junior) = 2/6 = 0.333
- **P(66-70K|Junior) = 0/6 = 0** (No junior has this salary!)

Step 3: Calculate Final Probabilities

For Senior:

$$P(Senior|X) = P(Senior) \times P(sales|Senior) \times P(31\text{-}35|Senior) \times P(66\text{-}70|Senior)$$

$$= (5/11) \times (1/5) \times (2/5) \times (2/5)$$

$$= 20/1375$$

$$\approx 0.0145$$

For Junior:

$$P(Junior|X) = P(Junior) \times P(sales|Junior) \times P(31\text{-}35|Junior) \times P(66\text{-}70|Junior)$$

$$= (6/11) \times (2/6) \times (2/6) \times (0/6)$$

$$= 0$$

Step 4: Final Classification

Compare probabilities:

- P(Senior|X) = 0.0145
- P(Junior|X) = 0

Predicted Class: SENIOR

Conclusion: The instance (sales, 31-35, 66K-70K) is classified as Senior because P(Senior|X) > P(Junior|X).

5.4 Variants of Naive Bayes

Type	Use Case	Assumption
GaussianNB	Continuous numeric data (age, salary, blood pressure)	Features follow Gaussian (normal) distribution
MultinomialNB	Text classification, word counts (spam detection)	Features represent counts or frequencies
BernoulliNB	Binary features (present/absent)	Features are binary (0 or 1)

6. K-Nearest Neighbors (KNN)

KNN is a simple, supervised machine learning algorithm used for both classification and regression. It makes predictions based on the K closest training examples in the feature space.

6.1 How KNN Works

5. Choose the number K (number of nearest neighbors)
6. Calculate the distance between the test instance and all training instances
7. Find the K nearest neighbors
8. **For Classification:** Vote for the most common class among the K neighbors
9. **For Regression:** Average the values of the K neighbors

6.2 Distance Calculation (Euclidean Distance)

$$\textbf{Distance} = \sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2]}$$

6.3 Complete Numerical Example

Problem: Predict Michelle's favorite singer. Michelle is 5 years old and Female.

Dataset:

Name	Age	Gender (0=M, 1=F)	Favorite Singer
Bill	9	0	Michael Jackson
Jane	4	1	Taylor Swift
Bob	11	0	Michael Jackson

Test Instance: Michelle (Age=5, Gender=1)

Step 1: Calculate Distances

Distance to Bill:

$\sqrt{[(5-9)^2 + (1-0)^2]} = \sqrt{[16 + 1]} = \sqrt{17} \approx 4.12$

Distance to Jane:

$\sqrt{[(5-4)^2 + (1-1)^2]} = \sqrt{[1 + 0]} = \sqrt{1} = 1.0$

Distance to Bob:

$\sqrt{[(5-11)^2 + (1-0)^2]} = \sqrt{[36 + 1]} = \sqrt{37} \approx 6.08$

Step 2: Find K Nearest Neighbors (K=1)

Sorted distances:

10. **Jane: 1.0 (CLOSEST)**
11. Bill: 4.12
12. Bob: 6.08

Predicted: Taylor Swift

Conclusion: With K=1, Michelle's favorite singer is predicted to be Taylor Swift (same as Jane's, who is the closest neighbor).

6.4 Choosing the Right K

- **Small K (e.g., K=1):** Sensitive to noise, may overfit
- **Large K:** Smoother decision boundaries, may underfit
- **Rule of Thumb:** K = $\sqrt{N}$ (where N is the number of training samples)
- Always use odd K for binary classification to avoid ties

7. One-R Classifier

One-R (One Rule) is a simple classification algorithm that generates one rule for each attribute in the training data, then selects the rule with the smallest error rate.

7.1 Algorithm

13. For each attribute:
    - Create a rule that predicts the class based on that attribute
    - Calculate the error rate of this rule
14. Select the rule with the lowest error rate

7.2 When to Use One-R

- Baseline model for comparison
- When you need a simple, interpretable model

- As a quick sanity check before building complex models

8. Flask Web Applications with ML Models

Flask is a lightweight Python web framework used to create web applications. It's perfect for deploying machine learning models as web services.

8.1 Basic Flask Application

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Flask!"

if __name__ == '__main__':
    app.run(debug=True)
```

Explanation:

- **app = Flask(__name__):** Creates a Flask application
- **@app.route('/'):** Defines the URL route (homepage)
- **def home():** Function that handles requests to that route
- **app.run(debug=True):** Starts the Flask development server

8.2 Flask with Machine Learning Model

Complete Example: Weather Prediction App

Step 1: Import Libraries

```
from flask import Flask, render_template, request
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
import joblib
```

Step 2: Prepare and Train Model

```
# Create sample dataset
data = {
    'Day': ['Sunny', 'Windy', 'Sunny', 'Windy'],
```

```
    'Temperature': ['Cool', 'Cool', 'Hot', 'Hot'],
    'Class': ['Play', 'Not Play', 'Not Play', 'Play']
}

df = pd.DataFrame(data)
X = df[['Day', 'Temperature']]
Y = df['Class']

# Encode features
onehot_encoder = OneHotEncoder()
X_encoded = onehot_encoder.fit_transform(X).toarray()

# Encode labels
label_encoder = LabelEncoder()
Y_encoded = label_encoder.fit_transform(Y)

# Train model
model = RandomForestClassifier()
model.fit(X_encoded, Y_encoded)

# Save model and encoders
joblib.dump(model, 'model.pkl')
joblib.dump(onehot_encoder, 'onehot.pkl')
joblib.dump(label_encoder, 'label.pkl')
```

Step 3: Create Flask Routes

```
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    # Get form data
    day = request.form['day']
    temperature = request.form['temperature']

    # Load model and encoders
    model = joblib.load('model.pkl')
    onehot = joblib.load('onehot.pkl')
    label = joblib.load('label.pkl')

    # Prepare input
    new_data = pd.DataFrame([[day, temperature]],
                            columns=['Day', 'Temperature'])
    new_encoded = onehot.transform(new_data).toarray()
```

```
    # Make prediction
    prediction = model.predict(new_encoded)
    result = label.inverse_transform(prediction)[0]

    return render_template('index.html',
                           prediction_text=f'Prediction:
{result}')
```

Key Concepts:

- **render_template():** Renders HTML templates from the 'templates' folder
- **request.form[]:** Accesses form data submitted via POST
- **joblib.load():** Loads saved ML models and encoders
- **inverse_transform():** Converts numeric prediction back to original label

9. Complete Code Examples

9.1 End-to-End ML Pipeline with Sklearn

```
# Import libraries
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Create dataset
data = {
    'Weather': ['Sunny', 'Rainy', 'Sunny', 'Cloudy', 'Rainy'],
    'Temperature': ['Hot', 'Cold', 'Hot', 'Cold', 'Cold'],
    'Play': ['Yes', 'No', 'Yes', 'Yes', 'No']
}
df = pd.DataFrame(data)

# Separate features and target
X = df[['Weather', 'Temperature']]
Y = df['Play']

# Encode features (One-Hot)
encoder = OneHotEncoder()
X_encoded = encoder.fit_transform(X).toarray()

# Encode target (Label Encoding)
```

```
label_encoder = LabelEncoder()
Y_encoded = label_encoder.fit_transform(Y)

# Split data (70% train, 30% test)
X_train, X_test, Y_train, Y_test = train_test_split(
    X_encoded, Y_encoded, test_size=0.3, random_state=42
)

# Train model
model = GaussianNB()
model.fit(X_train, Y_train)

# Make predictions
Y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(Y_test, Y_pred)
print(f'Accuracy: {accuracy * 100}%')
```

10. Practice Problems & Solutions

10.1 Bag of Words Practice

Problem: Create feature vectors using Bag of Words for the following sentences:

15. Sentence 1: He is moving to school
16. Sentence 2: She is not moving to college
17. Sentence 3: He likes school and college

Solution:

Step 1: Create Vocabulary

Vocabulary = [He, is, moving, to, school, She, not, college, likes, and]

Step 2: Create Feature Vectors

Sent	He	is	moving	to	school	She	not	college	likes	and
S1	1	1	1	1	1	0	0	0	0	0
S2	0	1	1	1	0	1	1	1	0	0
S3	1	0	0	0	1	0	0	1	1	1

10.2 Important Exam Tips

- **For Naive Bayes:** Always show all calculation steps. Remember to multiply all conditional probabilities.
- **For KNN:** Show distance calculations clearly. Remember to convert categorical data to numeric first.
- **For Bag of Words:** Create vocabulary from all sentences first, then count occurrences.
- **For Flask:** Understand the workflow: train model → save with joblib → load in Flask → make predictions.
- **For Python:** Know the difference between lists, dictionaries, sets, and tuples.

GOOD LUCK ON YOUR EXAM!

Study all numerical examples step-by-step

Practice coding examples hands-on